Is education the main driver of a high income? A CART classification tree from scratch on the Adult census data

Lucas Petrelluzzi, Manuel, Amine — Mathematics & Statistics for Machine Learning (A. Diallo), Albert School, MSc 1 — November 2026

Code: `github.com/Petrel971/ML-Project-Bootcamp` — data: `kaggle.com/datasets/uciml/adult-census-income`

Abstract. We implement a CART classification tree from scratch in NumPy (Gini impurity, vectorised exhaustive split search, greedy recursive growth, cost-complexity pruning, impurity-based importances) and use it to ask which census variables best predict an income above $50K. On a stratified test set our tree matches `sklearn.tree.DecisionTreeClassifier` exactly in accuracy (85.7 %), F1 (0.669) and agrees on 99.98 % of predictions, while training about 20× slower. Contrary to our hypothesis, education is not the first driver: being married comes first (importance 0.43, largest ablation loss), education second (0.22). We discuss why this is association, not causation, and where a single tree fails (minority-class recall, women, variance).

1. Problem and research question

The Adult dataset (UCI, extracted from the 1994 US Census) describes 48,842 adults with 14 attributes and a binary label: does the person earn more than $50K a year? It is a classic benchmark with a moderate class imbalance (23.9 % of >50K) and a mix of numeric and categorical variables. A common belief is that *education* is what drives income. We test it:

Research question. *Is education the main driver of earning more than $50K a year, or do other factors such as marital status, age and hours worked matter more?* Starting hypothesis: education is the number-one factor.

A decision tree suits the question: it handles mixed variables without scaling, and its rules and importances are readable. The brief also requires implementing the algorithm ourselves and comparing it with a library on the same splits; CART is short enough to be written and *verified* exactly against scikit-learn.

2. Data

Variable type	Variables
Numeric (5 kept)	age, education-num (1–16, diploma level), capital-gain, capital-loss, hours-per-week
Categorical (8)	workclass, marital-status, occupation, relationship, race, sex, native-country
Dropped (2)	**fnlwgt**: census sampling weight, not a property of the person. **education**: same information as education-num; keeping both would split the importance of education in two.

Missing values only affect three categorical variables: workclass (5.7 %), occupation (5.8 %) and native-country (1.8 %). They are *informative*: only 9.4 % of the rows with a missing occupation earn >50K versus 23.9 % overall. We therefore keep them as a category "Unknown" rather than dropping or imputing them. Taken one at a time, the variables most associated with income (Cramér's V) are relationship (0.45) and marital-status (0.45), then education (0.36), occupation (0.35), age (0.32) and hours (0.27).

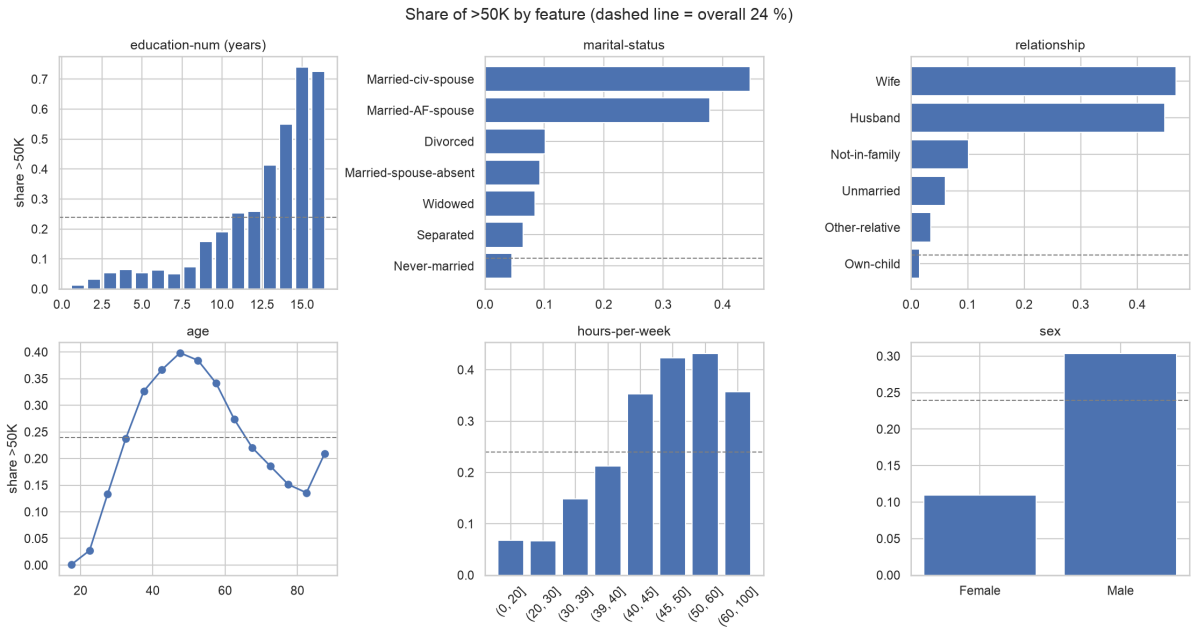


Figure 1 — Share of >50K by variable (dashed line: overall 24 %).

3. Method: CART from scratch

3.1 Model

A classification tree partitions the feature space into rectangles $R_1, \ldots, R_M$ (the leaves) with rules $x_j \le s$ at internal nodes. In leaf R_m the class proportions are $p_{mk} = (1/n_m) \Sigma_{i \in R_m} 1(y_i = k)$; the tree predicts $\text{argmax}_k\, p_{mk}$ and returns p_{mk} as probability.

3.2 Objective

Node impurity is measured by the Gini index (entropy is also implemented):

$$G = 1 - \Sigma_k p_k^2, \text{ 0 for a pure node, 0.5 at most with two classes}$$

A split sending n_L points left and n_R right is scored by the size-weighted impurity of its children; the best split minimises it, i.e. maximises the gain $G_{parent} - G_{split}$:

$$G_{split} = (n_L/n)\, G_L + (n_R/n)\, G_R$$

G is the chance that two points drawn at random in the node have different labels. Weighting by size prevents a split that isolates a single pure point from looking excellent. At the root of Adult $G = 1 - (0.761^2 + 0.239^2) = 0.364$. On exercise 3B.1 of the lecture (10 clients, root G = 0.48), "income > 30?" gives $G_{split} = 0.16$ and "age > 40?" 0.45; our search over all thresholds finds the income split (threshold 33, same partition).

3.3 Optimisation: split search

Between two consecutive distinct values a < b of a feature, all thresholds give the same partition, so we test the midpoints (a + b)/2, as scikit-learn does. A naive search splits and scores the data for every candidate: $O(d \cdot n^2)$. Our **vectorised** search sorts each feature once; moving to the next threshold moves one point from right to left, so the left class counts of all thresholds are the cumulative sums of the one-hot labels (right = total – left). All scores of a feature come from one NumPy expression, $O(d \cdot n \log n)$. On continuous data it is 36× faster than the naive version at n = 250 and 260× at n = 4,000.

3.4 Construction and stopping

As in Lecture 3B, the tree is grown greedily — best question now, no look-ahead — and recursively (finding the optimal tree is NP-hard). A node becomes a leaf when max_depth is reached, it has fewer than min_samples_split or 2·min_samples_leaf points, it is pure, no valid threshold exists, or the weighted decrease $(N_t/N)(G_t - G_{split})$ is below min_impurity_decrease. Optional cost-complexity pruning repeatedly collapses the node with the smallest $g(t) = (R(t) - R(T_t)) / (|T_t| - 1)$ while $g(t) \le \alpha$.

```
fit(X, y):  root = build(X, y, depth=0)
build(X, y, depth):
    node = leaf with class proportions of y
    if stopping rule: return node
    (j, s, score) = best_split(X, y)          # vectorised, O(d n log n)
    if no split or weighted decrease too small: return node
    node.rule = (x_j <= s)
    node.left  = build(X[x_j <= s], y[...], depth + 1)
    node.right = build(X[x_j >  s], y[...], depth + 1)
    return node
```

3.5 Prediction and importance

An observation goes down the tree until a leaf (we route all rows at once with boolean masks). The importance of feature j is Σ over nodes splitting on j of $(N_t/N)(G_t - (N_L/N_t)G_L - (N_R/N_t)G_R)$, normalised to 1. For one-hot encoded variables we sum the importances of their columns.

3.6 Verification

Exercise 3B.1 reproduced exactly, six automated tests (`uv run pytest`) and the checkpoints of notebook 01 check: known Gini values; naive = vectorised search; the same first split as scikit-learn; a perfect fit of the training set when the depth is unlimited; identical predictions and importances for depths 1–4; the same number of leaves after pruning. On Adult, predictions agree with scikit-learn at 100 % for depths 1–4 and 99.99 % at depths 5–6; the rare differences are **ties** (two one-hot columns with exactly the same Gini), broken by column order for us and by a random feature order in scikit-learn.

4. Experimental protocol

Step	Choice	Reason
Split	Stratified 60/20/20, seed 42	Train to learn, validation to tune, test used **once**; 23.9 % >50K in each set
Encoding	One-hot, columns fixed on train	Split first, then encode: no information from validation/test (no leakage)
Scaling	None	A tree compares a variable to a threshold: scale-invariant
Tuning	max_depth 1–20, min_samples_leaf 1–100	Selected by validation F1
Metrics	F1, precision, recall, ROC AUC, accuracy	A constant "≤50K" model already gets 76 % accuracy with F1 = 0
Comparison	Same splits, same hyperparameters	Ours vs DecisionTreeClassifier; also fit time and agreement rate

Train/validation/test sizes: 29,304 / 9,769 / 9,769 rows, 91 columns after encoding.

5. Results

5.1 Choosing the complexity

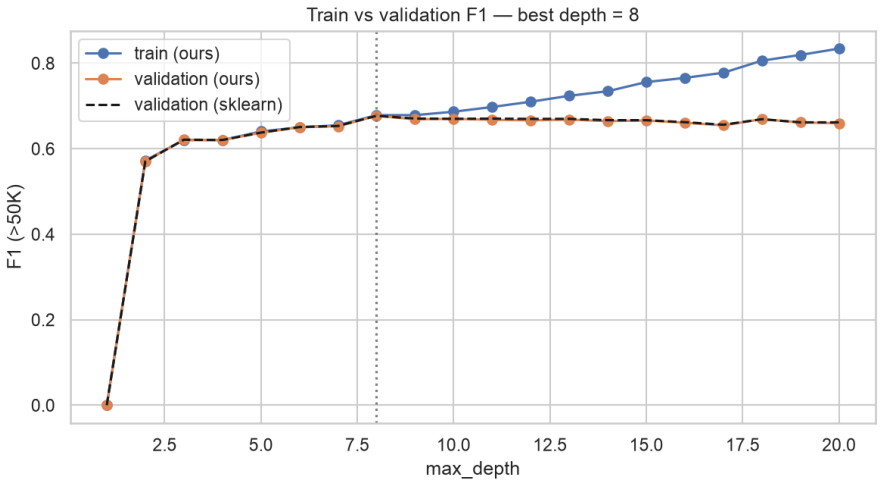


Figure 2 — Train and validation F1 by depth; our tree and scikit-learn overlap.

Validation F1 rises until depth 8 (0.677) and then stalls while train F1 keeps rising towards 0.8: deeper trees overfit (variance); shallower ones underfit (bias). Depth 1 has F1 = 0 although its AUC is 0.76: both children of the "married" split still have a ≤50K majority. min_samples_leaf = 5 slightly improves validation F1 (0.679) and AUC (0.910). **Chosen: max_depth = 8, min_samples_leaf = 5.**

5.2 Comparison on the test set

Model	Accuracy	Precision	Recall	F1	ROC AUC	Fit time
CART from scratch	0.857	0.751	0.602	0.669	0.906	≈ 1.0 s
scikit-learn	0.857	0.751	0.602	0.669	0.906	≈ 0.04 s
Always ≤50K	0.761	0	0	0	0.500	—

Both trees give the same prediction on 99.98 % of test rows (2 rows differ, caused by a tie) and the same metrics; the confusion matrices are identical (Figure 3). Our training is about 20× slower at every sample size from 2,000 to 29,304 rows; both grow roughly linearly in n. The gap is a constant factor: scikit-learn's splitter is compiled Cython working in place on sorted indices, while our Python recursion re-sorts features and copies sub-arrays at each node.

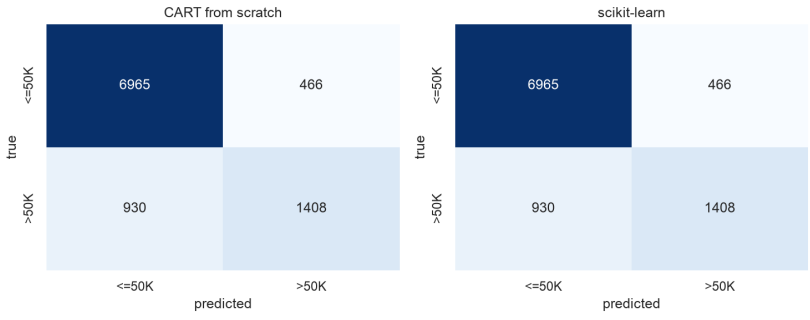


Figure 3 — Test confusion matrices (threshold 0.5).

6. Answer to the research question

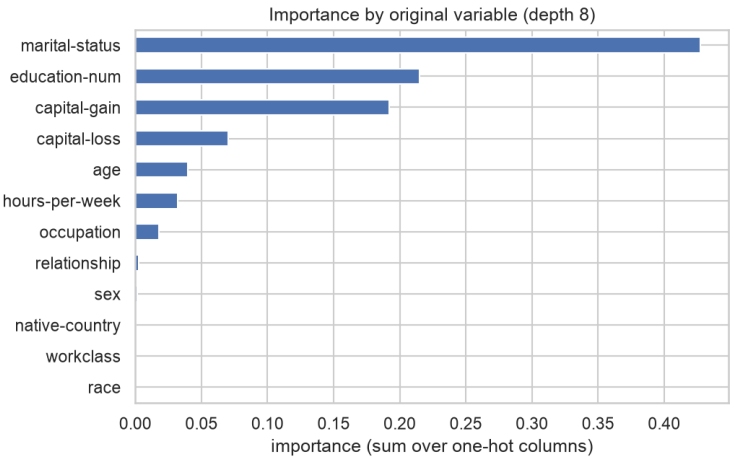


Figure 4 — Importance by original variable (depth-8 tree, one-hot columns summed).

Evidence	Family situation	Education	Others
Association alone (Cramér's V)	**0.45**	0.36	occupation 0.35, age 0.32, hours 0.27
First split	**married (civ-spouse)?**	—	—
Importance (depth 8)	**0.43**	0.22	capital-gain 0.19, capital-loss 0.07, age 0.04, hours 0.03
Ablation: validation AUC loss	**0.047**	0.013	capital 0.031, hours 0.007, age 0.005
Ablation: validation F1 loss	**0.122**	0.058	hours 0.048, capital 0.042, age ≈ 0
5 random splits	first split 5/5	2nd each time	—

The hypothesis is rejected. The family situation — being married and living with a spouse — is the strongest predictor by every measure: first split of the tree, twice the importance of education, and the largest loss when it is removed. Education is clearly the **second** factor, ahead of capital income; age and hours worked add little once marital status and education are known. The readable depth-3 tree tells the same story: unmarried people without capital gains have a 2–15 % chance of >50K depending on education; married people with at least an associate degree (education-num ≥ 12) reach 65 %.

Two subtleties. (1) relationship and marital-status carry the same information (Husband/Wife ⇔ married): the tree uses one and ignores the other, so removing either alone costs little; removing both costs the most. (2) Age has low importance not because it is irrelevant (V = 0.32) but because it is absorbed by marital status: importance measures what a variable adds *given the others*.

Association, not causation. Marriage does not cause a high income. In this 1994 sample, marital status is a proxy for age and career stage, for sex (married earners are mostly "Husband"), and possibly for household arrangements; capital-gain is closer to a consequence of wealth than a cause. Our conclusion is predictive: among these census variables, family situation is the best single predictor of a >50K income, and education the second.

7. Discussion: limits and when the model fails

Minority class. At the default threshold the tree misses 40 % of high earners (recall 0.60, precision 0.75). A threshold of 0.34, chosen on validation, raises test recall to 0.81 and F1 to 0.685 at the cost of precision (0.59) — the right threshold depends on the cost of each error.

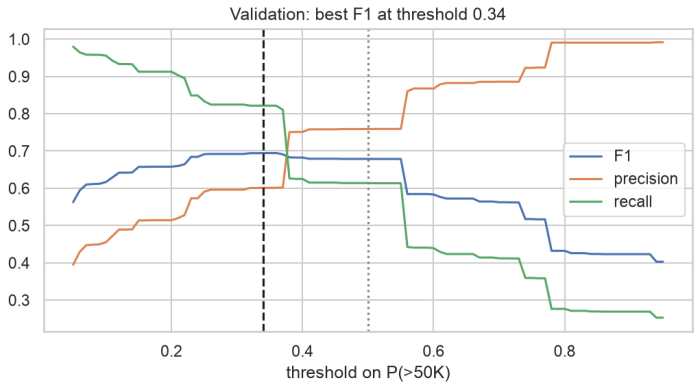


Figure 5 — Precision, recall and F1 by decision threshold (validation).

Errors by sex. Recall is 0.52 for women against 0.62 for men (F1 0.62 vs 0.68). Only 11 % of women in the test set earn >50K, and the model predicts 7.5 %: it learns and slightly amplifies the 1994 income gap. Any use of such a model for decisions about people would need a fairness analysis.

Variance. Over 5 random splits, the first split and the top importances are unchanged (marital 0.41–0.42, education 0.22–0.23), so our answer is robust; validation F1 varies from 0.638 to 0.659 and the deep, small leaves change. **Structure.** Axis-aligned cuts approximate smooth effects by staircases; greedy growth misses interactions invisible at the first step; impurity importance is computed on the training set and favours variables with many thresholds, which is why we added ablation. **Cost.** Our implementation is about 20× slower than scikit-learn for identical results.

Improvements. Ensembles reduce the weaknesses of one tree. For discussion we ran scikit-learn's random forest (300 trees: validation F1 0.681, AUC 0.920, lower variance) and gradient boosting (F1 0.719, AUC 0.931, lower bias) on the same splits, against 0.679 / 0.910 for our tree. Both reuse the split search we implemented as their building block; a natural extension is to build a random forest on our CART.

8. Conclusion

We implemented CART from scratch and verified it against scikit-learn: same splits, same predictions on 99.98 % of test rows, same metrics (accuracy 85.7 %, F1 0.669, AUC 0.906), with a 20× slower training. Used to answer our question, it shows that **education is not the main driver of a high income in the 1994 census: family situation comes first, education second**, a predictive association that reflects age, sex and household structure rather than a causal effect.

Reproducibility

`uv sync` installs the exact versions of `uv.lock` (Python ≥ 3.11, NumPy, pandas, scikit-learn, Matplotlib, Seaborn). `uv run pytest` runs the tests; then run `notebooks/01_cart_from_scratch.ipynb` and `notebooks/02_experiments.ipynb` top to bottom (the data is downloaded from OpenML on first run and cached in `data/`, not redistributed). Seed 42 everywhere; the model code is in `src/cart.py` (NumPy only), scikit-learn is used only for splitting, metrics and the comparison.

References

[1] Breiman, Friedman, Olshen, Stone (1984). *Classification and Regression Trees*. Wadsworth.
[2] Becker, Kohavi (1996). *Adult* [dataset]. UCI Machine Learning Repository, doi:10.24432/C5XW20.
[3] Hastie, Tibshirani, Friedman (2009). *The Elements of Statistical Learning*, ch. 9.2 and 15. Springer.
[4] Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12, 2825–2830.
[5] A. Diallo (2026). *Lecture 3B — Beyond Lines: Trees, SVMs and k-means*, Part I, and *Lab 1*. Mathematics & Statistics for ML, Albert School.